12. Matrix-Based Algorithms & Pattern Matching – Comprehensive Material

Module 1: Matrix Basics – Transpose & Searching

1.1 Matrix Transpose

Problem: Given an `m × n` matrix, return its transpose (an `n × m` matrix where rows become columns).

Approach: Create a new matrix `transpose[n][m]` and set `transpose[j][i] = matrix[i][j]`.

Code:

```
public static int[][] transpose(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    int[][] result = new int[n][m];
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            result[j][i] = matrix[i][j];
        }
    }
    return result;
}
```

Complexity: O(m × n) time, O(m × n) space (or in-place if square, but

careful).

1.2 Searching in a Sorted Matrix

Case 1: Matrix where rows and columns are sorted (each row sorted, each column sorted)

Problem: Search for a target value in an `m × n` matrix where each row is sorted left-to-right and each column is sorted top-to-bottom.

Optimal approach (O(m + n)): Start from the top-right corner (or bottom-left). If target == current, return true. If target < current, move left (since all elements below are larger). If target > current, move down (since all elements to the left are smaller).

Code:

```
public static boolean searchMatrix(int[][] matrix, int ta
    if (matrix == null || matrix.length == 0 || matrix[0]
    int m = matrix.length, n = matrix[0].length;
    int row = 0, col = n - 1;  // start at top-right
    while (row < m && col >= 0) {
        if (matrix[row][col] == target) return true;
        else if (target < matrix[row][col]) col--;
        else row++;
    }
    return false;
}
```

Visual Trace: matrix = [[1,4,7],[2,5,8],[3,6,9]], target = 5

- Start (0,2): 7 > 5 → col=1
- (0,1): 4 < 5 → row=1
- (1,1): 5 == 5 → true.

Case 2: Matrix where each row is sorted and the first element of

each row is greater than the last element of the previous row (fully sorted row-wise)

Problem: Treated as a 1-D sorted array of length `m*n`. Use binary search with index mapping: `row = mid / n`, `col = mid % n`.

Code:

```
public static boolean searchMatrixSorted(int[][] matrix,
    int m = matrix.length, n = matrix[0].length;
    int left = 0, right = m * n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int val = matrix[mid / n][mid % n];
        if (val == target) return true;
        else if (val < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

Module 2: Matrix DP – Advanced Problems

2.1 Maximal Square (LeetCode 221)

Problem: Given a 2-D binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

State: `dp[i][j]` = side length of the largest square ending at `(i,j)`.
Transition: If `matrix[i][j] == '1'`, then `dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])`. Else 0.

Code (space-optimized to 1D):

```
public static int maximalSquare(char[][] matrix) {
```

```
    int m = matrix.length, n = matrix[0].length;
    int[] dp = new int[n + 1];
    int maxSide = 0;
    int prev = 0; // dp[i-1][j-1]
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            int temp = dp[j];  // store old dp[j] (which
            if (matrix[i-1][j-1] == '1') {
                dp[j] = 1 + Math.min(Math.min(dp[j-1], dp
                maxSide = Math.max(maxSide, dp[j]);
            } else {
                dp[j] = 0;
            }
            prev = temp; // prev becomes dp[i-1][j-1] for
        }
    }
    return maxSide * maxSide;
}
```

Complexity: O(m × n) time, O(n) space.

2.2 Maximal Rectangle (LeetCode 85)

Problem: Given a binary matrix, find the largest rectangle containing only 1's.

Approach: Treat each row as the base of a histogram. Compute heights of consecutive 1's for each column. Then find the largest rectangle in histogram for each row.

Code (uses histogram helper):

```
public static int maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int m = matrix.length, n = matrix[0].length;
    int[] heights = new int[n];
    int maxArea = 0;
```

```
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == '1') heights[j]++;
            else heights[j] = 0;
        }
        maxArea = Math.max(maxArea, largestRectangleArea(
    }
    return maxArea;
}

// Helper: largest rectangle in histogram
private static int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int maxArea = 0;
    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.p
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

Complexity: O(m × n) time, O(n) space.

Module 3: Tries – Construction, Insertion, Search, Autocomplete

3.1 Trie Node and Operations

Trie Node: Contains an array of children (size 26 for lowercase letters) and a boolean flag `isEnd`.

Code – Full Implementation:

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd;
}

class Trie {
    private TrieNode root;

    public Trie() { root = new TrieNode(); }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = searchPrefix(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return searchPrefix(prefix) != null;
    }

    private TrieNode searchPrefix(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return null;
            node = node.children[idx];
        }
        return node;
```

```
    }

    // Autocomplete: return all words with given prefix
    public List<String> autocomplete(String prefix) {
        List<String> result = new ArrayList<>();
        TrieNode node = searchPrefix(prefix);
        if (node == null) return result;
        dfs(node, new StringBuilder(prefix), result);
        return result;
    }

    private void dfs(TrieNode node, StringBuilder sb, Lis
        if (node.isEnd) result.add(sb.toString());
        for (char c = 'a'; c <= 'z'; c++) {
            int idx = c - 'a';
            if (node.children[idx] != null) {
                sb.append(c);
                dfs(node.children[idx], sb, result);
                sb.deleteCharAt(sb.length() - 1);
            }
        }
    }
}
```

Visual Trace: Insert "cat", "car", "dog". Search "car" → true. Autocomplete for "ca" → ["car", "cat"].

Complexity: Insert/Search O(L) where L = word length. Autocomplete O(number of nodes in subtree).

Module 4: Pattern Matching Algorithms

4.1 Knuth-Morris-Pratt (KMP) Algorithm

Problem: Find all occurrences of a pattern `P` in a text `T`.

Core Idea: Preprocess pattern to build **LPS** (Longest Proper Prefix which is also Suffix) array. Use it to skip characters when a mismatch occurs.

LPS Construction:

```
public static int[] computeLPS(String pattern) {
    int m = pattern.length();
    int[] lps = new int[m];
    int len = 0;  // length of previous longest prefix su
    int i = 1;
    while (i < m) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}
```

KMP Search:

```
public static List<Integer> kmpSearch(String text, String
    int n = text.length(), m = pattern.length();
    int[] lps = computeLPS(pattern);
    List<Integer> indices = new ArrayList<>();
    int i = 0; // index for text
    int j = 0; // index for pattern
    while (i < n) {
        if (pattern.charAt(j) == text.charAt(i)) {
            i++; j++;
```

```
            }
            if (j == m) {
                indices.add(i - j);
                j = lps[j - 1];
            } else if (i < n && pattern.charAt(j) != text.cha
                if (j != 0) j = lps[j - 1];
                else i++;
            }
        }
        return indices;
    }
```

Visual Trace: text = "ABABDABACDABABCABAB", pattern = "ABABCABAB". LPS = [0,0,1,2,0,1,2,3,4]. Search finds at index 10.

Complexity: O(n + m) time, O(m) space.

4.2 Z-Algorithm

Problem: Find all occurrences of a pattern in O(n + m) time using a Z-array.

Z-array: `Z[i]` = length of the longest substring starting at `i` that matches the prefix of the string.

Algorithm: Concatenate pattern + "$" + text. Compute Z-array. Positions where Z[i] == pattern length indicate a match.

Code:

```
public static List<Integer> zAlgorithm(String text, Strin
    String combined = pattern + "$" + text;
    int n = combined.length();
    int[] Z = new int[n];
    int L = 0, R = 0;
    for (int i = 1; i < n; i++) {
        if (i <= R) {
            Z[i] = Math.min(R - i + 1, Z[i - L]);
```

```
        }
        while (i + Z[i] < n && combined.charAt(Z[i]) == c
            Z[i]++;
        }
        if (i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }
    List<Integer> result = new ArrayList<>();
    int m = pattern.length();
    for (int i = m + 1; i < n; i++) {
        if (Z[i] == m) result.add(i - m - 1);
    }
    return result;
}
```

Visual Trace: pattern = "aba", text = "ababa". Combined = "aba$ababa". Z at positions where Z[i] == 3 indicate matches.

Complexity: O(n + m) time, O(n + m) space.

4.3 Rabin-Karp Algorithm (Rolling Hash)

Problem: Find pattern using a hash function. Compute hash of pattern and compare with hash of each window of text. On hash match, verify character-by-character (to avoid collisions).

Code (using a simple hash with base and mod):

```
public static List<Integer> rabinKarp(String text, String
    int n = text.length(), m = pattern.length();
    if (m > n) return new ArrayList<>();
    int d = 256; // number of characters
    int q = 101; // prime modulus (small for demo)
    int h = 1;
    // compute (d^(m-1)) % q
```

```
    for (int i = 0; i < m - 1; i++) h = (h * d) % q;

    int patternHash = 0, textHash = 0;
    for (int i = 0; i < m; i++) {
        patternHash = (patternHash * d + pattern.charAt(i
        textHash = (textHash * d + text.charAt(i)) % q;
    }

    List<Integer> indices = new ArrayList<>();
    for (int i = 0; i <= n - m; i++) {
        if (patternHash == textHash) {
            // verify
            if (text.substring(i, i + m).equals(pattern))
        }
        if (i < n - m) {
            textHash = (textHash - text.charAt(i) * h) %
            textHash = (textHash * d + text.charAt(i + m)
            if (textHash < 0) textHash += q;
        }
    }
    return indices;
}
```

Complexity: Average O(n + m), worst-case O(n*m) if many hash collisions.

Variation: Use double hashing or mod larger prime to reduce collisions.

4.4 Multi-Pattern Matching – Aho-Corasick (Concept-level)

Problem: Find all occurrences of a set of patterns in a text simultaneously.

Concept: Build a **Trie** of all patterns. Add **failure links** (like KMP's LPS) to enable transitions when a mismatch occurs. Each node also has an `output` list of patterns that end at that node. Then traverse the text, following transitions (treating failure links as fallback) and collect outputs.

Key Idea: It is an extension of KMP to multiple patterns using a Trie with automaton transitions.

Complexity: Construction O(total length of patterns). Searching O(text length + number of occurrences).

Implementation is non-trivial; used in practice for large-scale string matching.

Module 5: Hands-on Problems

Practice Exercises

1. **Search in a 2D Matrix II** – LeetCode 240 (row/col sorted).
2. **Search in a Sorted Matrix** – LeetCode 74 (fully sorted).
3. **Maximal Square** – LeetCode 221.
4. **Maximal Rectangle** – LeetCode 85.
5. **Word Search** – LeetCode 79 (backtracking).
6. **Word Search II** – LeetCode 212 (Trie + DFS).
7. **Implement Trie** – LeetCode 208.
8. **Longest Common Prefix** – Using Trie.
9. **KMP Search** – Implement and test.
10. **Rabin-Karp** – Implement and test.
11. **Z-Algorithm** – Implement.
12. **Aho-Corasick** – Understand and implement a simple version.

Summary Table

Topic	Key Technique	Complexity
Matrix Transpose	Nested loops	O(m*n)

Search in Sorted Matrix (row/col)	Two pointers from corner	O(m+n)
Search in Fully Sorted Matrix	Binary search on flattened array	O(log(m*n))
Maximal Square	DP with min of neighbors	O(m*n)
Maximal Rectangle	Histogram DP per row	O(m*n)
Trie	Insert/Search/ Autocomplete	O(length)
KMP	LPS preprocessing	O(n+m)
Z-Algorithm	Z-array construction	O(n+m)
Rabin-Karp	Rolling hash	O(n+m) average
Aho-Corasick	Trie + failure links	O(total patterns + text)